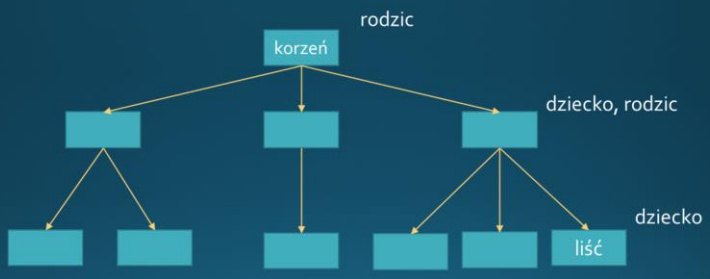
Sortowanie

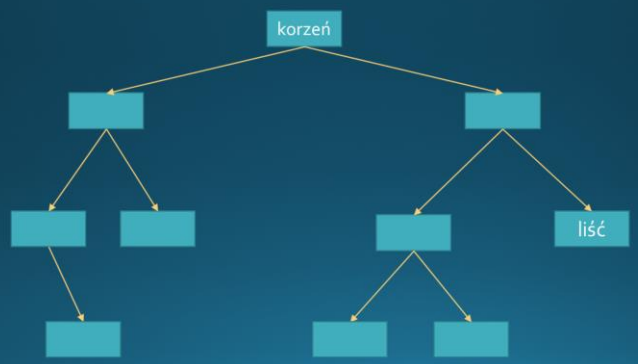
1. Sortowanie przez wstawiania (ang. insertion sort),
2. Sortowanie przez wybieranie (ang. selection sort),
3. Sortowanie bąbelkowe (ang. bubble sort),
4. Metoda Shella (ang. shellsort),
5. Kopcowanie (ang. heapsort),
6. Sortowanie szybkie (ang. quicksort),
7. Sortowanie przez scalanie (ang. mergesort).

Zadania do samodzielnej pracy: https://projecteuler.net

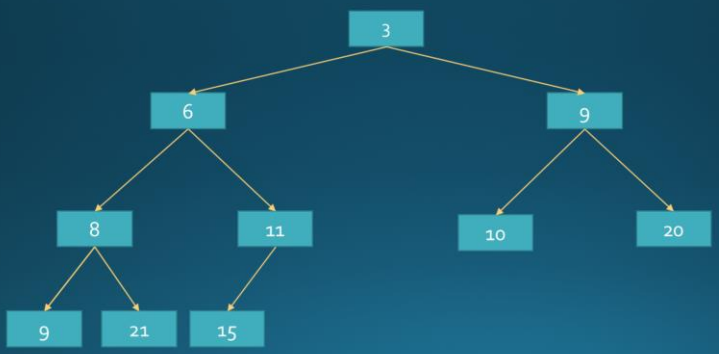
Drzewo



Drzewo binarne



Kopiec (stóg, sterta)



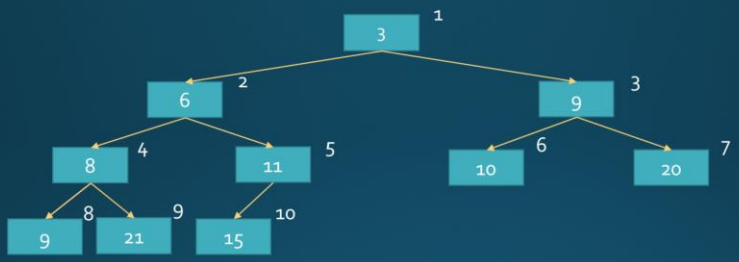
Kopiec

Kopiec – drzewo binarne, w którym wartość związana z dowolnym wierzchołkiem jest mniejsza lub równa wartościom związanym z następnikami (potomkami)

Kopiec zupełny – wszystkie warstwy muszą być po kolei zapełniane

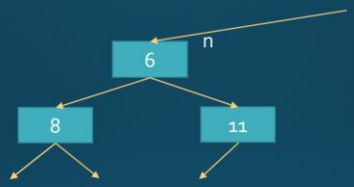
Korzeń kopca przechowuje najmniejszą wartość

Reprezentowanie kopca w tablicy



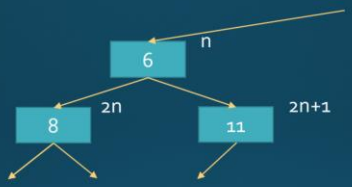
	1	2	3	4	5	6	7	8	9	10
	3	6	9	8	11	10	20	9	21	15

Reprezentowanie kopca w tablicy



	1	2	3	4	5	6	7	8	9	10
	3	6	9	8	11	10	20	9	21	15

Reprezentowanie kopca w tablicy



indeksy dzieci węzła n: 2n (lewe dziecko), 2n+1 (prawe dziecko)

indeks rodzica dzieca n: n/2

	1	2	3	4	5	6	7	8	9	10
	3	6	9	8	11	10	20	9	21	15

Kopiec - implementacja

Dane:

- const int HEAP_MAX = 100;
- int heap_size = 0;
- double heap[HEAP_MAX + 1];

Funkcje:

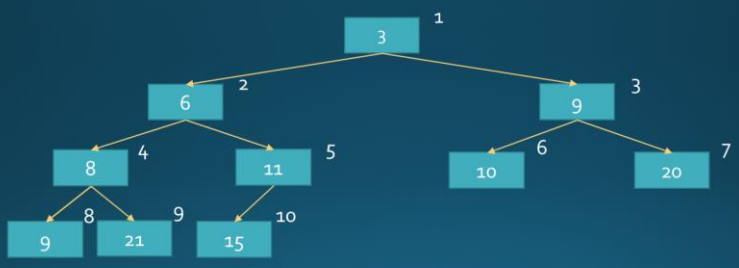
- void insert(double value, double heap[], int& heap_size, const int HEAP_MAX);
- void remove_min(double heap[], int& heap_size);
- double min(const double heap[], int heap_size);
- int min_child(int parent, const double heap[], int heap_size);

Kopiec – funkcja min

Funkcja min zwraca wartość minimalną przechowywaną w kopcu.

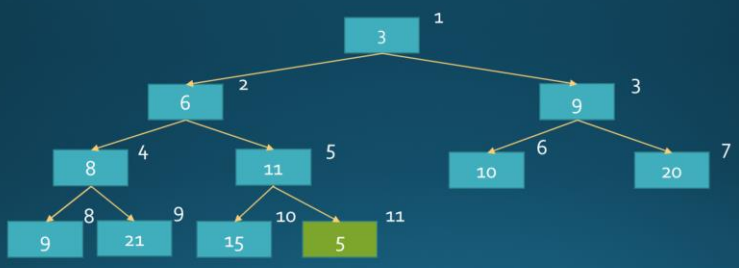
```
double min(const double heap[], int heap_size)
{
  return heap[1];
}
```

Kopiec – wstawianie (funkcja insert)



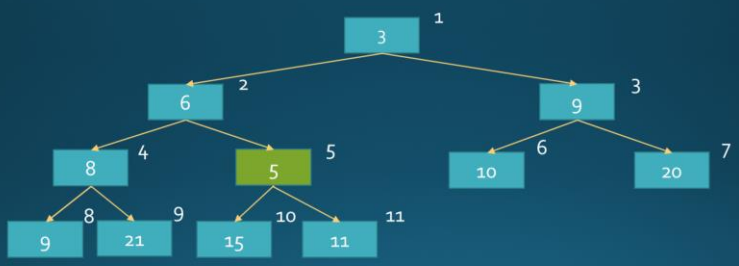
chcemy wstawić wartość 5

Kopiec – wstawianie (funkcja insert)



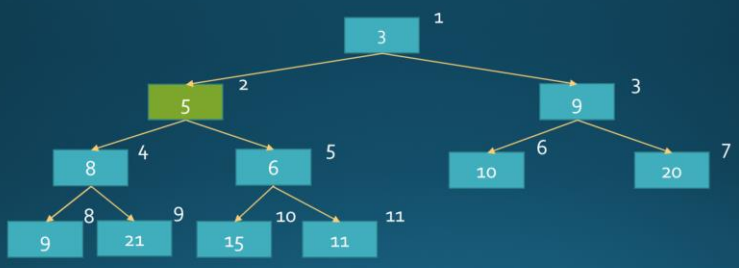
chcemy wstawić wartość 5

Kopiec – wstawianie (funkcja insert)



chcemy wstawić wartość 5

Kopiec – wstawianie (funkcja insert)



chcemy wstawić wartość 5

Kopiec – wstawianie (funkcja insert)

```
void insert(double value, double heap[], int& heap_size, const int HEAP_MAX) {
  int child = ++heap_size;
  int parent = child / 2;
  while (parent && heap[parent] > value) {
     heap[child] = heap[parent];
     child = parent;
     parent /= 2;
  }
  heap[child] = value;
}
```

Kopiec – wstawianie (funkcja insert)

Ile czasu potrzeba do wstawienia wartości do kopca?

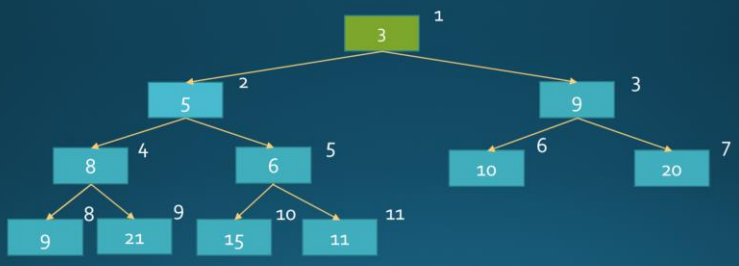
W najgorszym przypadku należy przejść przez wszystkie warstwy.

Ile jest warstw?

$$\log_2 n$$

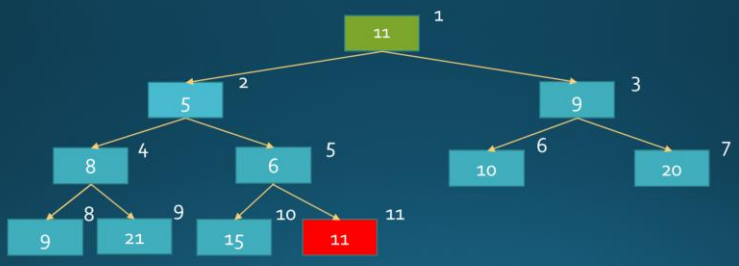
Złożoność jest więc logarytmiczna.

Usuwanie (funkcja remove_min)



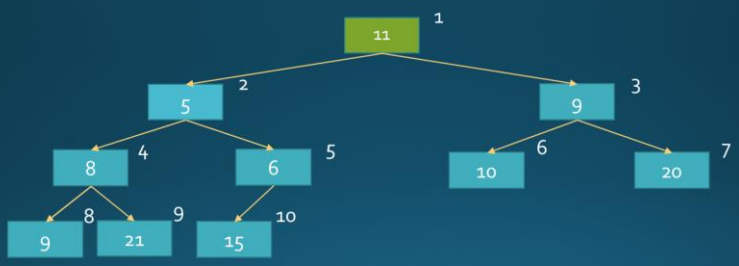
chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)



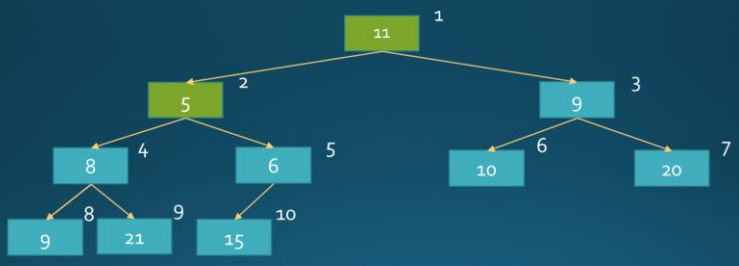
chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)



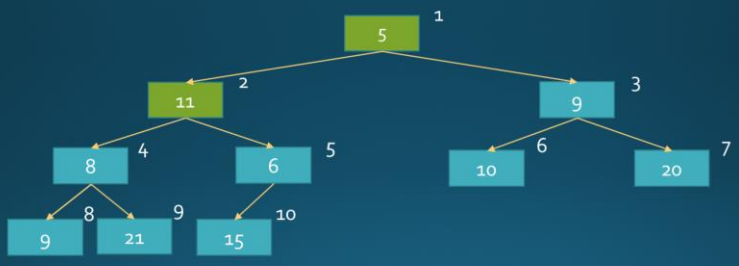
chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)



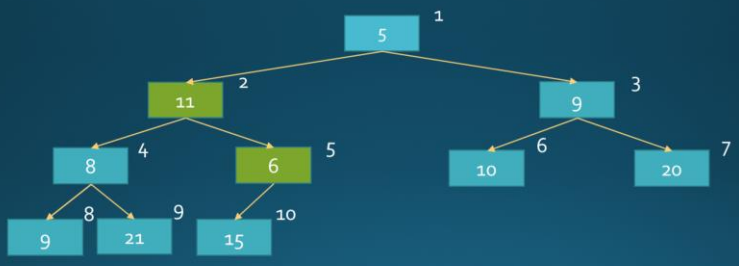
chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)



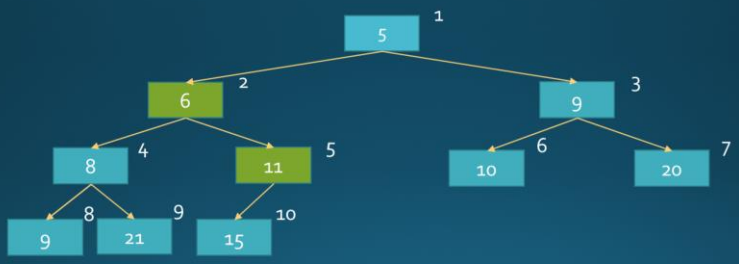
chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)



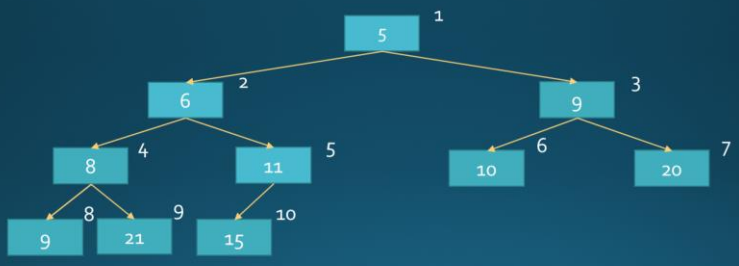
chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)



chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)



chcemy usunąć wartość najmniejszą

Usuwanie (funkcja remove_min)

```
void remove_min(double heap[], int& heap_size)
{
  double last = heap[heap_size--];
  int temp = 1;
  int min_child_idx = min_child(1); //zwraca 0 jeśli brak dzieci
  while (min_child_idx && heap[min_child_idx] < last) {
    heap[temp] = heap[min_child_idx];
    temp = min_child_idx;
    min_child_idx = min_child(x);
  }
  heap[temp] = last;
}
```

Usuwanie (funkcja remove_min)

Ile czasu potrzeba do usunięcia najmniejszej wartości do kopca?

W najgorszym przypadku należy przejść przez wszystkie warstwy.

Ile jest warstw?

$$\log_2 n$$

Złożoność jest więc logarytmiczna.

Znajdowanie mniejszego dziecka

```
int min_child(const int heap[], int parent, int heap_size)
{
  int left  = 2 * parent;
  int right = left + 1;
  if (left >  heap_size) return 0;
  if (right > heap_size || heap[left] < heap_[right])
     return left;
  return right;
}
```

Sortowanie przez kopcowanie

Algorytm:

- utwórz kopiec z danych do posortowania,
- pobierz wartość minimalną z kopca i wstaw do tablicy,
- usuń wartość minimalną z kopca,
- powtórz kroki 2 i 3 aż kopiec nie zostanie opróżniony.

Sortowanie przez kopcowanie

```
void heapsort(double data[], int size)
{
  const int HEAP_MAX = 100;
  if (size > HEAP_MAX) return;
  double heap[HEAP_MAX];
  int heap_size = 0;

  for (int i = 0; i < size; ++i)
    insert(data[i], heap, heap_size, HEAP_MAX);

  for (int i = 0; i < size; ++i) {
    data[i] = min(heap, heap_size);
    remove_min(heap, heap_size);
  }
}
```

Sortowanie przez kopcowanie

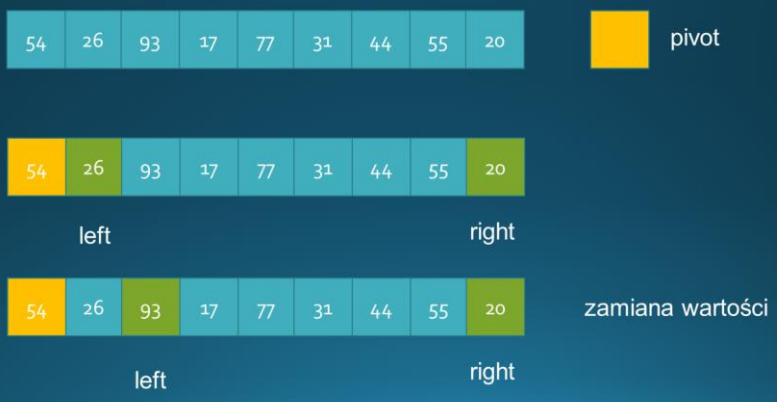
Złożoność obliczeniowa etapów sortowania:

- wstawianie do kopca wykonywane jest n razy – złożoność $O(n \log n)$,
- usuwanie wartości minimalnej wykonywane jest n razy – złożoność $O(n \log n)$,
- znajdowanie mniejszego dziecka wykonywane jest n razy – złożoność $O(n)$

Złożoność obliczeniowa sortowania:

$$O(n \log n)$$

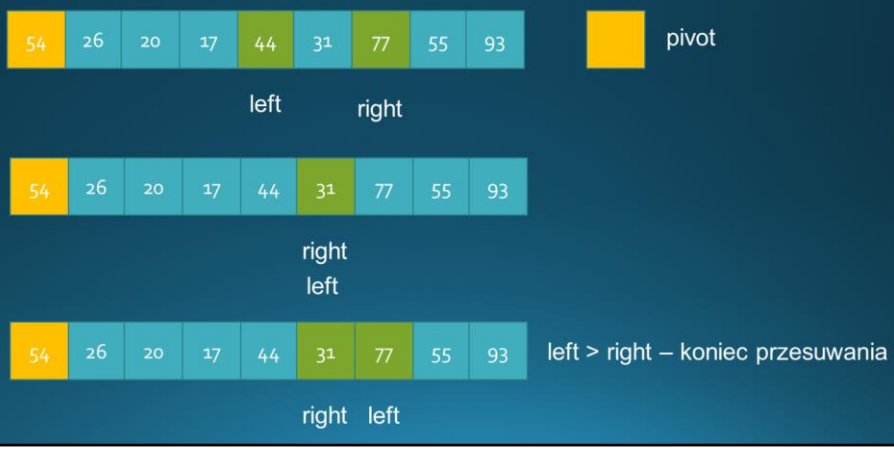
Sortowanie szybkie



Sortowanie szybkie



Sortowanie szybkie



Sortowanie szybkie



Sortowanie szybkie

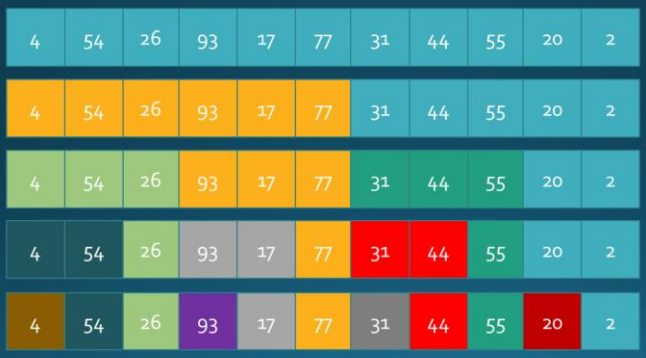
```
void quicksort(double data[], int first, int last) {
  if (first >= last) return;
  double pivot = data[first];
  int left = first;
  int right = last - 1;
  while (true) {
    do  left++; while (left <= last  && data[left] < pivot);
    do  right--; while (data[right] > pivot);
    if (left > right) break;
    swap(data[left], data[right]);
  }
  swap(data[first], data[right]);
  quicksort(data, first, right - 1);
  quicksort(data, right + 1, last);
}
```

Sortowanie szybkie

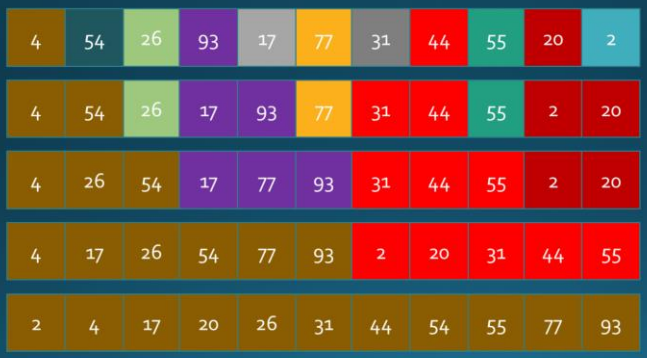
Średnia złożoność obliczeniowa: $O(n \log n)$

Złożoność najgorszego przypadku: $O(n^2)$

Sortowanie przez scalanie



Sortowanie przez scalanie

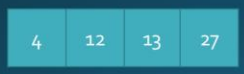


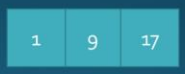
Proste scalanie zbiorów

Dana jest posortowana rosnąco tablica A o rozmiarze N.

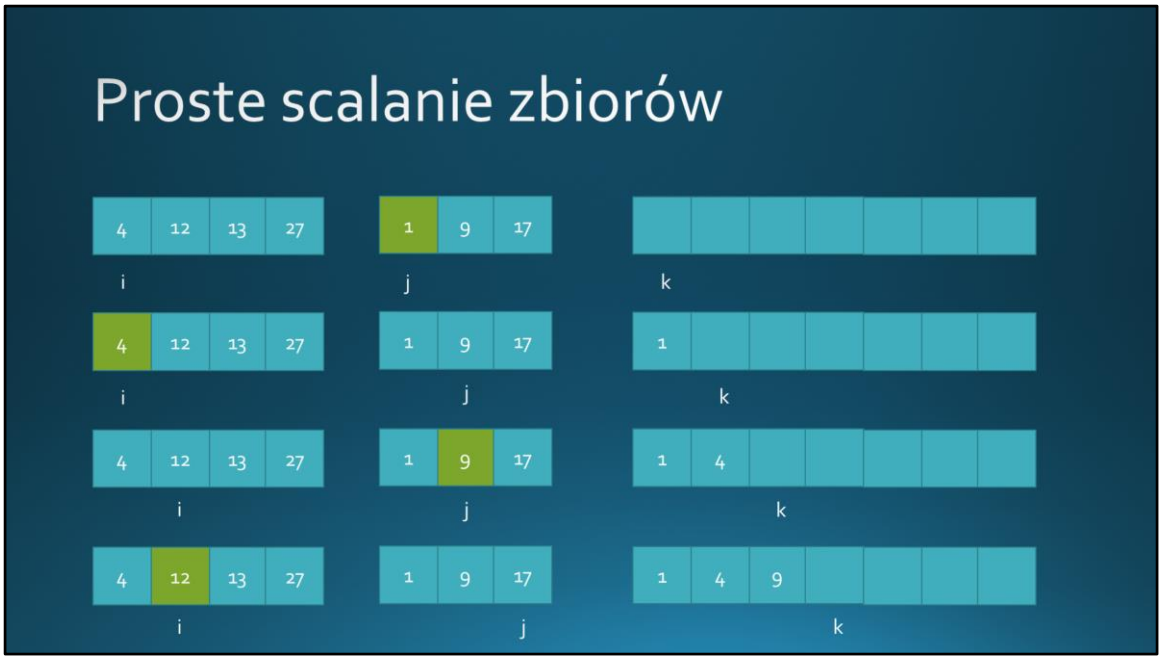
Dane jest posortowana rosnąco tablica B o rozmiarze M.

Należy „połączyć" obie tablice w tablicę C o rozmiarze N+M tak aby była ona posortowana rosnąco.

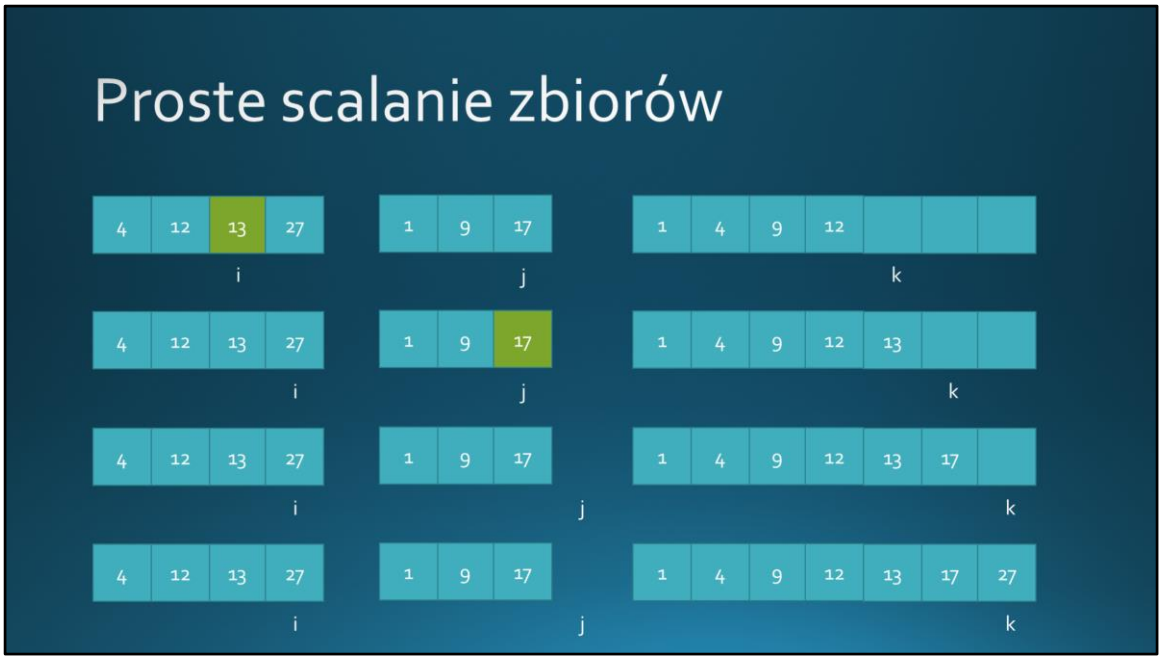




Proste scalanie zbiorów



Proste scalanie zbiorów



Proste scalanie zbiorów

```
void mergeAB(int C[], int A[], int sizeA, int B[], int sizeB)
{
  for (int i = 0, j = 0, k = 0; k < sizeA + sizeB; k++) {
     if (i == sizeA) { C[k] = B[j++]; continue; }
     if (j == sizeB} { C[k] = A[i++]; continue; }
     C[k] = (A[i] < B[j]) ? A[i++] : B[j++];
  }
}
```

Dwie instrukcje warunkowe „if” mogą spowalniać program.

Scalanie zbiorów

Dana jest posortowana rosnąco tablica A o rozmiarze N.

Dane jest posortowana rosnąco tablica B o rozmiarze M.

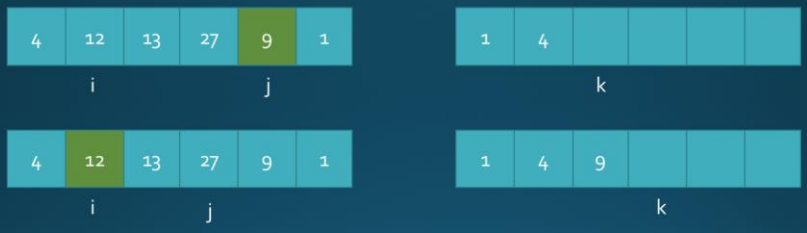
Należy „połączyć" obie tablice w tablicę C o rozmiarze N+M tak aby była ona posortowana rosnąco.



Scalanie zbiorów



Scalanie zbiorów



Od tego momentu kolejne wartości będą porównywane z wartością największą. Nie trzeba pilnować zakresów tablic składowych.

Scalanie zbiorów

```
void merge(int A[], int first, int middle, int last) {
  int i, j, k, Aux[1000];
  for (i = middle + 1; i > first; i--) Aux[i – 1] = A[i – 1];
  for (j = middle; j < last; j++) Aux[last + m – j] = A[j + 1];

  for (int r = first; r <= last; r++)
    if (Aux[j] < Aux[i])
      A[k] = Aux[j--];
    else
      A[k] = Aux[i++];
}
```

Sortowanie przez scalanie

```
void mergesort(int A[], int first, int last) {
  if (last <= first) return;

  int middle = (first + last) / 2;
  mergesort(A, first, middle);
  mergesort(A, middle+1, last);
  merge(A, first, middle, last);
}
```

Sortowanie przez scalanie

- Złożoność obliczeniowa: $O(n \log n)$,
- Wada – konieczność stosowania dodatkowej tablicy.

Porównanie złożoności

	best	average	worst	memory	stable
insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	no
bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Shell	-	-	$O(n^{1,5})$	$O(1)$	no
heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	no
quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	no
mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes